Strojno učenje

Nevronske mreže in odločitvena drevesa

Jakob Jerič, 28221068
Fakulteta za matematiko in fiziko, Univerza v Ljubljani
jj06142@student.uni-lj.si

6. september 2025

Naloga

Na spletni učilnici je na voljo material (koda, vzorci) za ločevanje dogodkov Higgsovega bozona od ostalih procesov ozadja. V naboru simuliranih dogodkov je 18 karakteristik (zveznih kinematičnih lastnosti), katerih vsaka posamezno zelo slabo loči 'signal' od ozadja, z uporabo BDT ali (D)NN, pa lahko tu dosežemo zelo dober uspeh. Na predavanjih smo si ogledali glavne aspekte pomembne pri implementaciji ML, kot so uporaba ustreznih spremenljivk (GIGO), učenje in prekomerno učenje (training/overtraining), vrednotenje uspeha metode kot razmerje med učinkovitostjo (efficiency) in čistostjo (precision) vzorca (Receiver Operating Characteristic, ROC). Določi uspešnost obeh metod (in nariši ROC) za nekaj tipičih konfiguracij BDT in DNN, pri čemer:

- Študiraj vpliv uporabljenih vhodnih spremenljivk - kaj, če vzamemo le nekatere?
- Študiraj BDT in NN in vrednoti uspešnost različnih nastavitev, če spreminjaš nekaj konfiguracijskih parametrov (npr. število perceptronov in plasti nevronskih mrež pri DNN in število dreves pri BDT).

Uvod

Uporaba algoritmov strojnega učenja (Machine Learning, ML) je v znanosti danes že rutina. Ločimo tri osnovne pristope:

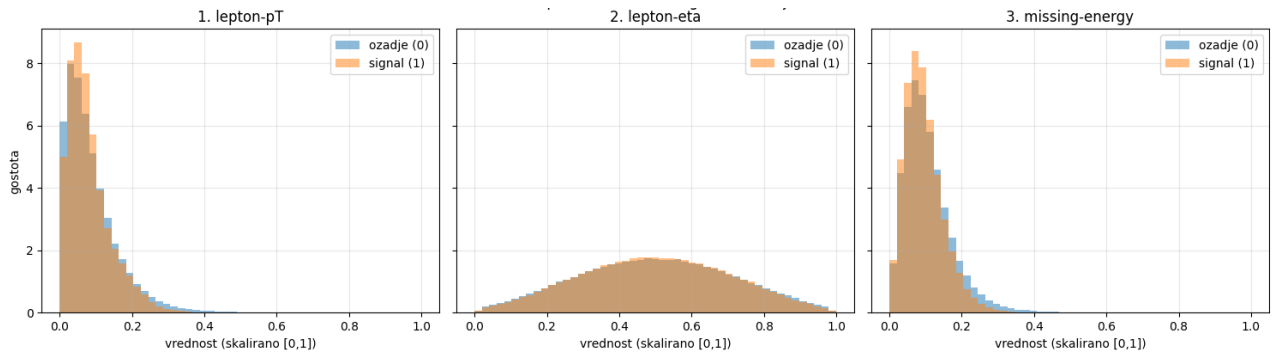
- **Nadzorovano učenje** (supervised):
  - *klasifikacija* (razvrščanje primerov v razrede),
  - *regresija* (napoved zveznih količin).
- **Nenadzorovano učenje**: samodejno iskanje struktur in skupin v podatkih.
- **Okrepljeno učenje** (reinforcement): učenje na podlagi nagrad in kazni.

V fiziki delcev (in v tej nalogi) praviloma uporabljamo nadzorovano učenje: za prepoznavo signalov novih pojavov/delcev ali za ekstrakcijo netrivialnih funkcijskih odvisnosti.

Glavna prednost ML pred »klasičnimi« metodami je, da kompleksne probleme v več dimenzijah učinkovito razdrobi na enostavne gradnike:

- v večdimenzijskih prostorih je težko uganiti zaprto analitično obliko, medtem ko so fleksibilne aproksimacije (npr. zlepki) praktične in natančne;
- pri filtriranju velikih vzorcev z mnogo lastnostmi (npr. dogodki na LHC) je ročno težko izbrati optimalne spremenljivke, upoštevati korelacije in najti kombinacijo, ki najbolje loči signal od ozadja.

Na spodnji sliki so prikazane porazdelitve prvih treh (od 18) značilk za naš primer - ločevanje signala od ozadja.



Slika 1: Porazdelitve signala in ozadja za izbrane prve značilke učnega nabora.

Matematični okvir. Predpostavimo učno množico

$$\mathcal{D} = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^{N},$$

kjer je $\mathbf{x}_k = (x_k^1, \ldots, x_k^M) \in \mathbb{R}^M$ *vektor značilk* (npr. kinematične veličine), $\mathbf{y}_k$ pa *ciljna oznaka* (pri binarni klasifikaciji $\mathbf{y}_k \in \{0, 1\}$); pare $(\mathbf{x}_k, \mathbf{y}_k)$ obravnavamo kot neodvisne vzorce iz neznane porazdelitve P. Iščemo preslikavo h, ki minimizira pričakovano izgubo

$$\mathcal{L}(h) = \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim P}\, L\big(\mathbf{y}, h(\mathbf{x})\big) \approx \frac{1}{N}\sum_{k=1}^{N} L\big(\mathbf{y}_k, h(\mathbf{x}_k)\big),$$

kjer je L funkcija izgube (pri klasifikaciji navzkrižna entropija, pri regresiji MSE). Po učenju $h(\cdot)$ za nov vhod $\hat{\mathbf{x}}$ poda napoved $\hat{\mathbf{y}} = h(\hat{\mathbf{x}})$. Funkcije h so sestavljene iz preprostih gradnikov z nekaj prostimi parametri (utežmi), kar vodi v večji nabor neznank in zahteven minimizacijski problem.

Odločitvena drevesa in BDT. Osnovni gradnik je stopničasta funkcija $H(x_i - t_i) = \mathbb{1}\{x_i > t_i\}$, kjer je x_i izbrana značilka, t_i prag. Iz zaporedja takih odločitev sestavimo drevo; napoved lahko shematično zapišemo kot

$$h(\mathbf{x}) = \sum_{j=1}^{J} a_j\, \mathbb{1}\{\text{pogoj}_j(\mathbf{x})\}.$$

Nadgradnja so *pospešena* drevesa (BDT), kjer uporabimo uteženo množico zaporedno naučenih dreves (npr. AdaBoost, gradientno pospeševanje).

Nevronske mreže (DNN). Osnovni gradnik je perceptron

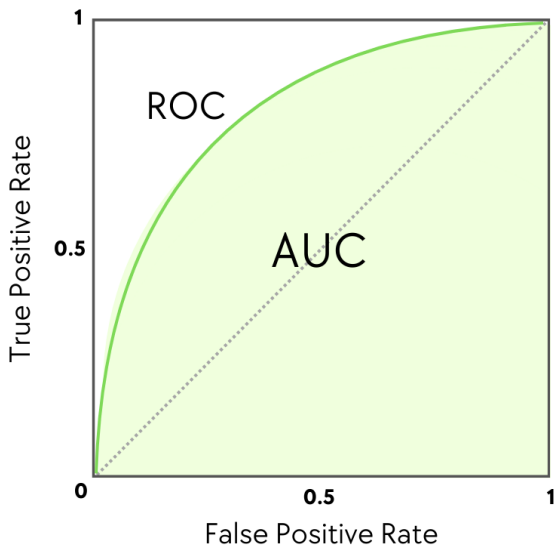
$$h_{w,b}(\mathbf{x}) = \theta(\mathbf{w}^\top \mathbf{x} + b),$$

kjer je $\mathbf{x}$ celoten vektor značilk (npr. naših 18 spremenljivk), $\mathbf{w}$ uteži, b odmik, θ pa aktivacijska funkcija (npr. sigmoid, tanh, ReLU). Z zlaganjem plasti dobimo mrežo, ki preslika $\mathbf{x}$ v napoved $\hat{y}$; pri binarni klasifikaciji na izhodu uporabimo sigmoid in izgubo *binary cross-entropy*. Prenaučenost omejujemo z regularizacijo (L2, dropout) ali zgodnjim ustavljanjem.

Ocenjevanje (ROC/AUC). Za izhod $s = h(\mathbf{x})$ izberemo prag t in izračunamo

$$\mathrm{TPR}(t) = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad \mathrm{FPR}(t) = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}.$$

Z višanjem/zniževanjem t dobimo ROC (TPR proti FPR), *AUC* pa je ploščina pod krivuljo (1 = popoln, 0.5 = naključen klasifikator).



Slika 2: Primer ROC krivulje.

Odločitvena drevesa (BDT)

Eno drevo. Eno odločitveno drevo si lahko predstavljamo kot zaporedje *if–else* pravil, ki na podlagi vhodnih značilk postopoma delijo množico vzorcev v manjše in bolj homogene skupine. V vsakem notranjem vozlišču se izbere značilka x_i in prag t, ki zagotavljata najboljšo delitev glede na izbrano *funkcijo izgube*. Matematično to pomeni, da se poišče tak (i, t), ki povzroči največje zmanjšanje izgube pri razdelitvi podatkov na levo in desno vejo.

Razcepljanje se ponavlja do največje dovoljene *globine* d. Drevo globine d ima največ 2^d končnih vozlišč oziroma *listov*. Plitva drevesa opisujejo le grobe odločitvene meje, globlja pa omogočajo podrobnejše razločevanje, a so tudi bolj nagnjena k prenaučenosti. Vsak list predstavlja skupino vzorcev, ki so se znašli v enaki regiji prostora značilk.

Ko so listi enkrat določeni, je treba vsakemu prirediti napoved. V najenostavnejšem primeru bi to bil delež signalnih vzorcev v listu. V praksi pa se vrednosti listov izberejo tako, da najbolj zmanjšajo funkcijo izgube za vse vzorce v listu. To vodi do realnih vrednosti na *logit* skali:

$$F_{\text{list}} \in \mathbb{R},$$

ki lahko zavzamejo pozitivne, negativne ali ničelne vrednosti. Končna klasifikacija novega vzorca poteka tako, da ga drevo po zaporedju *if–else* pravil pripelje do enega od listov, kjer prejme ustrezno konstanto F_{list}. Ta se nato pretvori v verjetnost razreda s pomočjo logistične funkcije

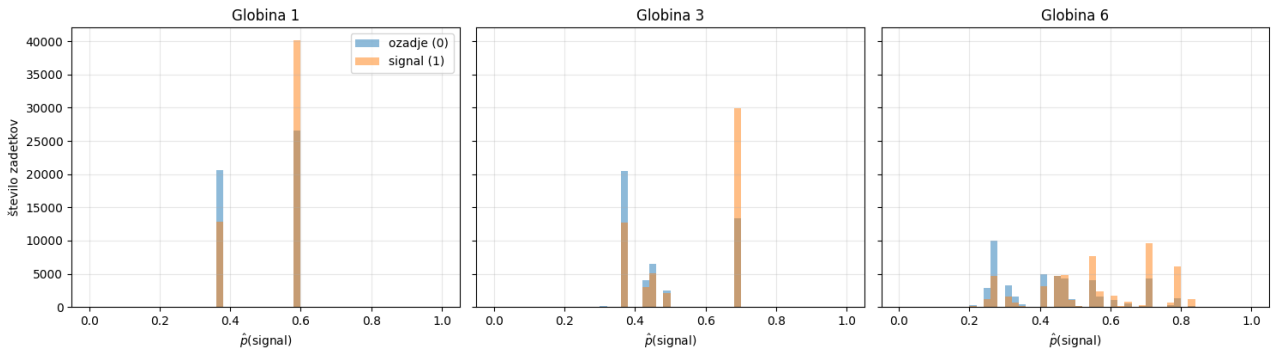
$$\hat{p} \;=\; \sigma(F_{\text{list}}) \;=\; \frac{1}{1 + e^{-F_{\text{list}}}}.$$

Na ta način eno samo odločitveno drevo implementira hierarhičen sistem logičnih pravil, ki razdeli prostor značilk na disjunktne regije (liste), in vsaki regiji pripiše konstanto, izbrano tako, da čim bolj zmanjša izgubo. Globina drevesa določa, koliko takih regij se oblikuje in kako podrobna je razločitev podatkovnega prostora.

(a) Drevo globine 1 (2 lista). (b) Drevo globine 2 (4 listi).

Slika 3: Primer odločitvenih dreves različnih globin: (a) globina 1, (b) globina 2.

Na spodnji sliki lahko vidimo, kako tri drevesa različnih globin ločijo signal od ozadja.



Slika 4: Primer razločevanja signala od ozadja z drevesi različnih globin.

Več dreves skupaj (pospeševanje). Namesto enega zelo globokega drevesa zgradimo zaporedje plitvih dreves, katerih prispevki se *seštevajo*. Označimo z $F_m(\mathbf{x})$ trenutno oceno po m-tem koraku in z $f_m(\mathbf{x})$ prispevek m-tega drevesa; model je aditiven

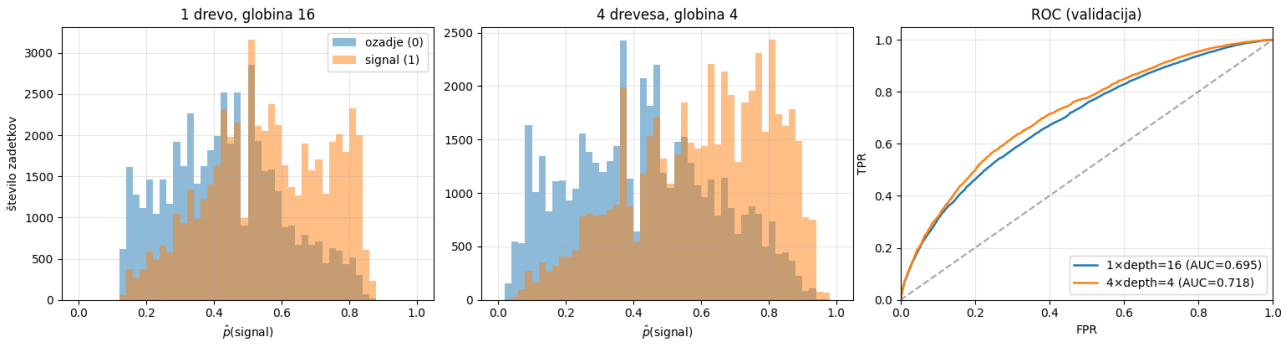
$$F_0(\mathbf{x}) = \text{konst.}, \qquad F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu\, f_m(\mathbf{x}) \quad (m = 1, \ldots, M),$$

kjer je $0 < \nu \leq 1$ *learning rate*. Vsako drevo po *if–else* pravilih razdeli prostor v liste; za vzorec $\mathbf{x}$ je prispevek $f_m(\mathbf{x})$ enak konstanti lista $F^{(m)}_{\text{list}}$, izbrani tako, da zniža izgubo na vzorcih v tem listu. Ključno: *koliko različnih ocen F lahko nastane*? Pri enem drevesu globine d je največ 2^d različnih konstant (po ena na list). Pri zaporedju M dreves, kjer ima m-to drevo L_m listov, celoten model razdeli prostor na preseke teh listov; v vsakem preseku je končna ocena

$$F_M(\mathbf{x}) \;=\; F_0 \;+\; \nu \sum_{m=1}^{M} F^{(m)}_{\text{list}_m(\mathbf{x})},$$

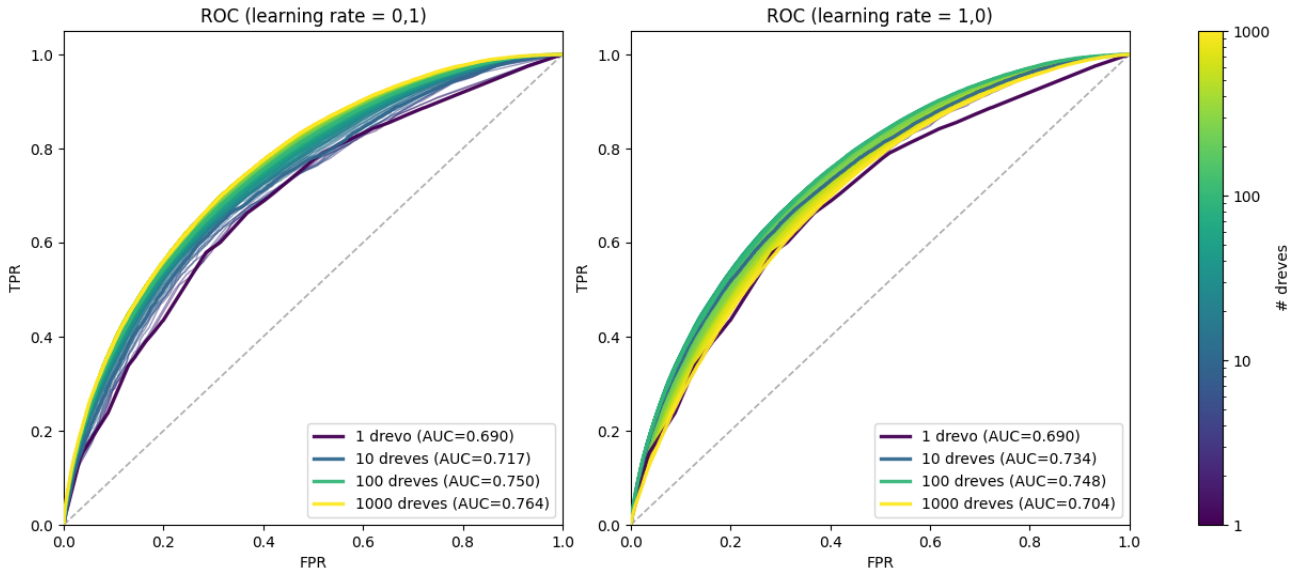
zato je število različnih končnih ocen navzgor omejeno z največ $\prod_{m=1}^{M} L_m$ (če imajo vsa drevesa globino d, je to največ 2^{Md}). V praksi je teh ocen manj, ker so nekateri preseki prazni ali nedosegljivi. Končno verjetnost razreda izračunamo šele na koncu iz skupne ocene z logistično funkcijo $\hat{p}(\mathbf{x}) = \sigma(F_M(\mathbf{x})) = \frac{1}{1+e^{-F_M(\mathbf{x})}}$. Tako kombinacija več plitvih dreves zgradi bogate, a stabilne odločitvene meje: vsaka *kombinacija poti* skozi drevesa določa eno od možnih stalnih ocen v prostoru značilk.

Na spodnji sliki vidimo, da je "učinkovitost" šestnajstih *if–else* pravil boljša (večja vrednost AUC) če jih uporabimo v štirih drevesih globine 4, kot v enem drevesu globine 16.



Slika 5: Primerjava enega zelo globokega drevesa (1×globina 16 - AUC = 0,695) z več plitvimi drevesi (4×globina 4 - AUC = 0,718).

Na grafu na naslednji strani so prikazane ROC krivulje za različno število dreves v BDT. Levi graf je imel *learning rate* 0,1, največjo ploščino pod krivuljo pa je dosegel pri 767 drevesih - AUC = 0,764; desni pa je z *learning rate* 1,0, največjo ploščino pod krivuljo dosegel pri 65 drevesih - AUC = 0,751.



Slika 6: ROC krivulje za različne nastavitve števila dreves in *learning rate*.

Pri večjem *learning rate* je AUC pričakovano na začetku z večanjem števila dreves hitreje rasel, vendar pa je bila maksimalna vrednost manjša.

Nevronske mreže (DNN)

Perceptron. Perceptron je najpreprostejša nevronska enota za binarno klasifikacijo: sprejme vhodni vektor značilk $\mathbf{x} \in \mathbb{R}^M$ in izračuna linearno kombinacijo vhodov z utežmi in odmikom

$$z = \mathbf{w}^\top \mathbf{x} + b,$$

kjer so $\mathbf{w}$ uteži in b odmik.

Pri *trdem perceptronu* je izhod enostavna diskretna odločitev

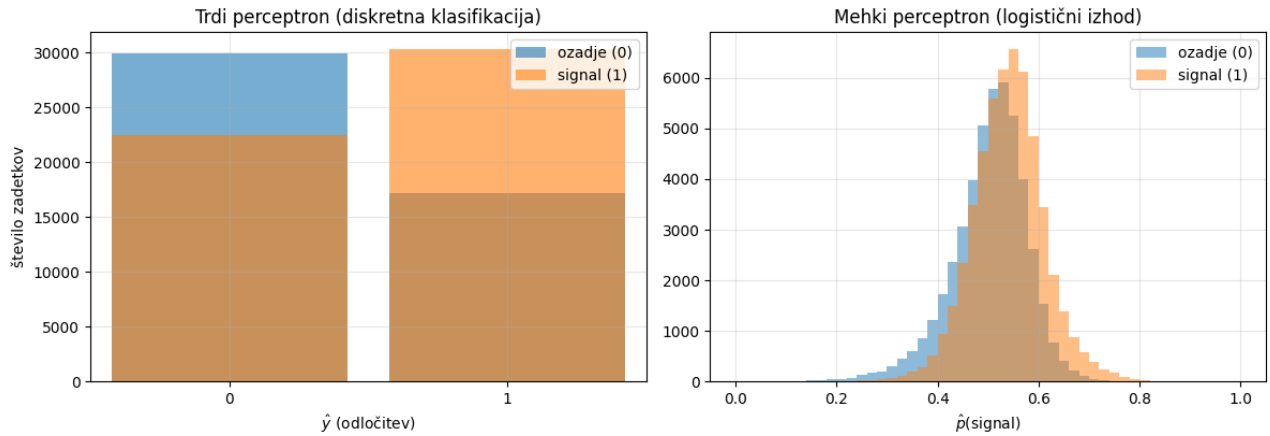
$$\hat{y} = \begin{cases} 1, & z > t, \\ 0, & z \leq t, \end{cases}$$

kar pomeni, da model vrne razred 1, če linearna kombinacija preseže prag t, sicer razred 0. Odločitvena meja je hiperploščina $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = t\}$, ki linearno razcepi prostor značilk na dva dela.

Pri *mehkem (logističnem) perceptronu* pa se linearna kombinacija prenese skozi logistično funkcijo

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}},$$

kar interpretiramo kot verjetnost, da vzorec pripada signalu ($y = 1$); končna odločitev je nato podana s pragom na tej verjetnosti. Na spodnjem grafu vidimo, kakšne izhode daje trdi (levo) in mehki (desno) perceptron na validacijskem vzorcu.

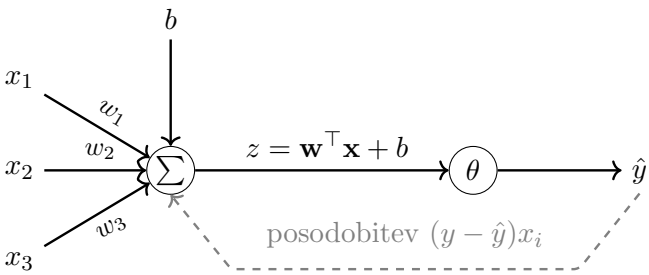


Slika 7: Izhodi trdega (levo) in mehkega (desno) perceptrona na validacijskem vzorcu.

Učenje perceptrona pomeni prilagajanje parametrov $\mathbf{w}, b$. Pri trdi različici se uporablja preprosto pravilo posodabljanja

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}, \qquad b \leftarrow b + \eta\,(y - \hat{y}),$$

kjer je η učni korak, y prava oznaka vzorca (0 za ozadje, 1 za signal), $\hat{y}$ pa napoved perceptrona. Pri mehki različici pa parametre optimiziramo z gradientnim spustom na funkciji izgube.



Slika 8: Perceptron: vhodi x_i se utežijo z w_i, seštejejo z odmikom b v z, aktivacija θ da izhod $\hat{y}$. Siva povratna puščica shematično prikazuje učenje (posodobitev uteži).

Nevronska mreža. En sam perceptron lahko loči le podatke, ki so linearno ločljivi, saj odločitev temelji na hiperploščini

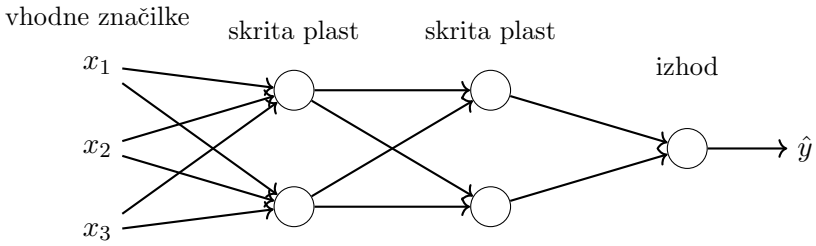
$$\mathbf{w}^\top \mathbf{x} + b = 0,$$

ki prostor značilk razdeli na dva dela. Če razredi niso linearno ločljivi, en perceptron ne more najti ustrezne odločitvene meje.

Nevronska mreža to omejitev preseže tako, da vhodne značilke zaporedno preslika skozi več plasti perceptronov z nelinearnimi aktivacijami. Za plast ℓ zapišemo

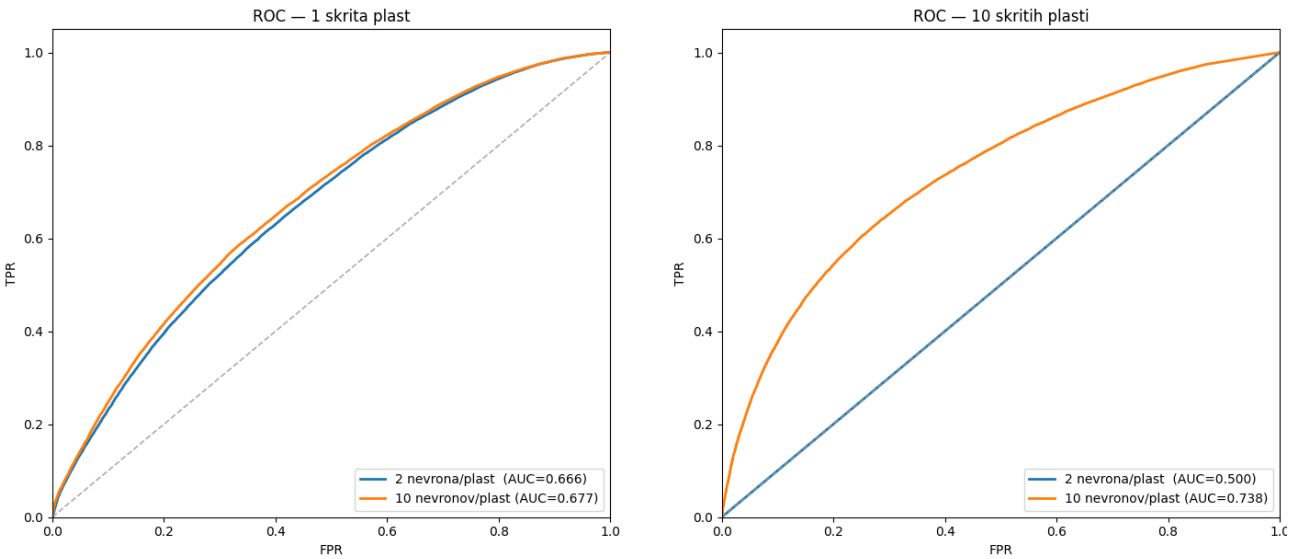
$$\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{a}^{(\ell)} = \phi^{(\ell)}\big(\mathbf{z}^{(\ell)}\big), \quad \mathbf{a}^{(0)} = \mathbf{x},$$

kjer so $W^{(\ell)}$ matrike uteži, $\mathbf{b}^{(\ell)}$ odmiki, $\phi^{(\ell)}$ pa nelinearne aktivacije. Ker se linearne preslikave med seboj skrčijo v eno linearno preslikavo, je ključna nelinearnost $\phi^{(\ell)}$: s sestavljanjem več takih transformacij mreža oblikuje kompleksne, nelinearne odločitvene meje, zato lahko loči tudi podatke, ki jih en sam perceptron — z eno hiperploščino — ne more.



Slika 9: Shema nevronske mreže: zaporedne plasti nelinearnih perceptronov tvorijo kompleksne odločitvene meje.

Na spodnjem grafu lahko primerjamo ROC krivulje za 4 različne konfiguracije nevronskih mrež: po dve z eno skrito plastjo (levo) in po dve z desetimi skritimi plastmi (desno). V obeh primerih smo imeli plasti s po 2 in s po 50 nevroni.



Slika 10: ROC krivulje za različne konfiguracije nevronskih mrež.

Najbolje se odreže model z *10 plastmi in 10 nevroni na plast* (najvišji AUC). Presenetljivo pa je *10-plastni model z 2 nevronoma na plast* skoraj naključen (AUC ≈ 0.5). Razlog: že prva plast skrči 18 vhodov na le 2 dimenziji (ozko grlo), zato se izguba informacije v globini ne more nadomestiti, dodatne zaporedne nelinearnosti pa otežijo učenje (oslabljen/izginjajoč gradient, "ugasle" ReLU). Pri

eni skriti plasti z 2 nevronoma je rezultat boljši, ker enostavna projekcija v 2D z eno nelinearnostjo še vedno zajame del ločljivosti podatkov.

Učenje nevronske mreže. Naj bo $\theta = \{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$ množica parametrov mreže in z h_θ označimo mrežo s temi parametri. Učenje poteka na mini-serijah $\mathcal{B}$ velikosti B, kjer minimiziramo povprečno izgubo

$$\mathcal{L}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell\big(y_i,\, h_\theta(\mathbf{x}_i)\big), \qquad g(\theta; \mathcal{B}) = \nabla_\theta \mathcal{L}_{\mathcal{B}}.$$

V naprejni poti velja

$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{a}^{(\ell)} = \phi\big(\mathbf{z}^{(\ell)}\big),$$

kjer so $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}^{(\ell)}, \mathbf{z}^{(\ell)}, \mathbf{a}^{(\ell)} \in \mathbb{R}^{n_\ell}$. Gradient izračunamo z verižnim zakonom: uvedemo

$$\delta^{(\ell)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} \in \mathbb{R}^{n_\ell}, \qquad \delta^{(L)} = \hat{p} - y \;\; \text{(pri sigmoid + navzkrižni entropiji)},$$

in nazaj po plasteh dobimo

$$\delta^{(\ell)} = \big(W^{(\ell+1)}\big)^\top \delta^{(\ell+1)} \;\odot\; \phi'\big(\mathbf{z}^{(\ell)}\big),$$

kjer sta oba faktorja vektorja dolžine n_ℓ in $\odot$ pomeni elementno množenje (ker ϕ deluje po komponentah). Od tod sledita

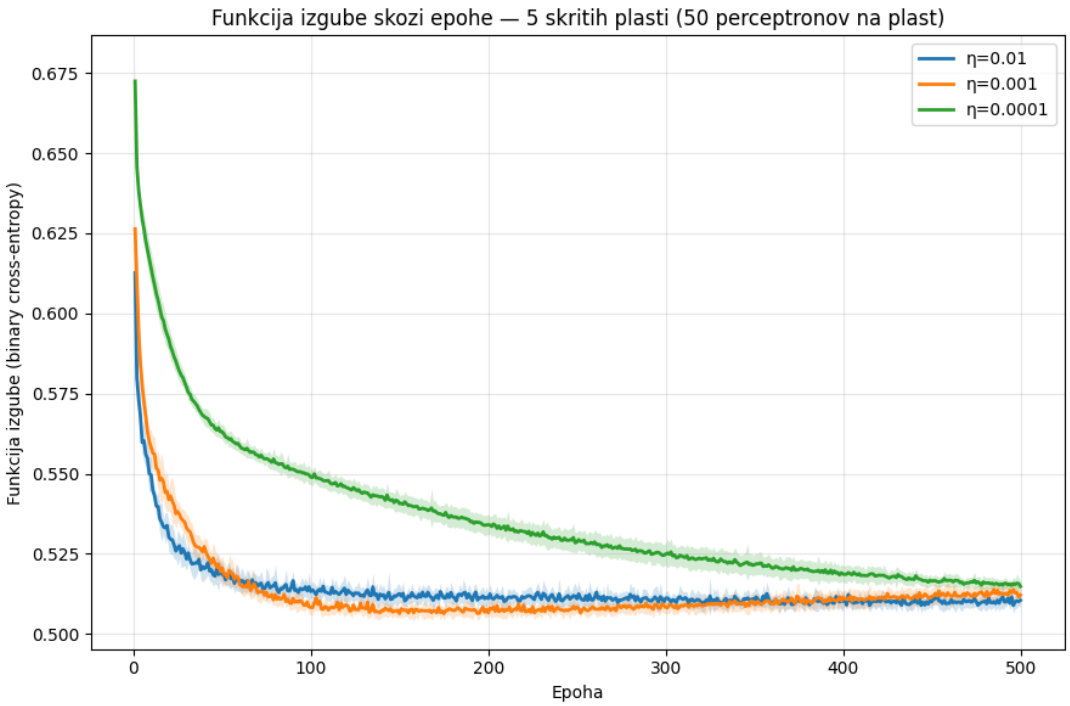
$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \big(\mathbf{a}^{(\ell-1)}\big)^\top, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)}.$$

Parametre posodobimo z gradientnim spustom

$$\theta_{t+1} = \theta_t - \eta\, g(\theta_t; \mathcal{B}),$$

kjer $\eta > 0$ določa velikost koraka: premajhen η učenje upočasni, prevelik povzroča nihanje. Ker učimo po mini-serijah, zaporedje približno $\lceil N/B \rceil$ takih posodobitev, ki skupaj porabijo vseh N učnih primerov, imenujemo *epoha*; po vsaki epohi običajno preverimo uspešnost na validacijskem vzorcu.

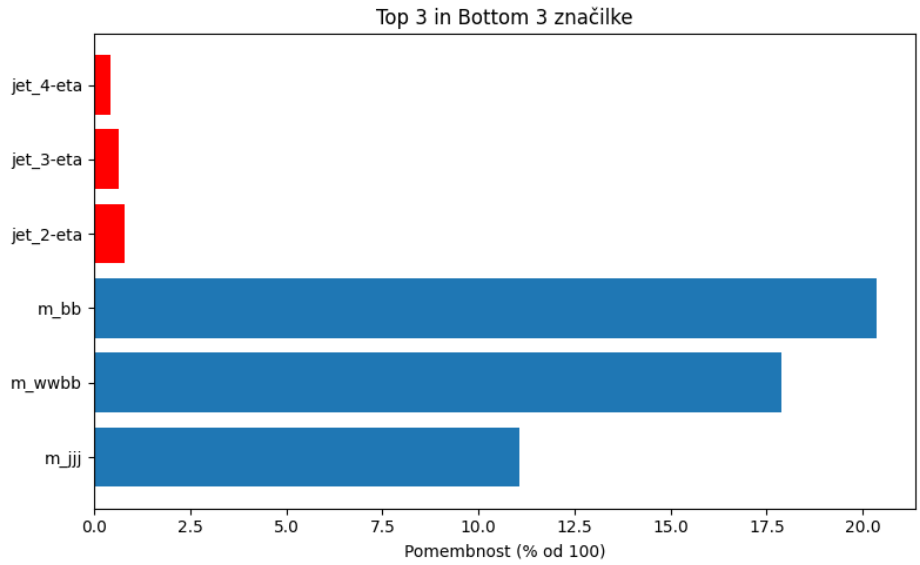
Na spodnjem grafu vidimo kako se skozi epohe spreminja funkcija izgube za različne velikosti koraka η. Naš model ima 5 plasti s po 50 nevroni na plast.



Slika 11: Funkcija izgube na učnem in validacijskem vzorcu skozi epohe za različne velikosti koraka η.

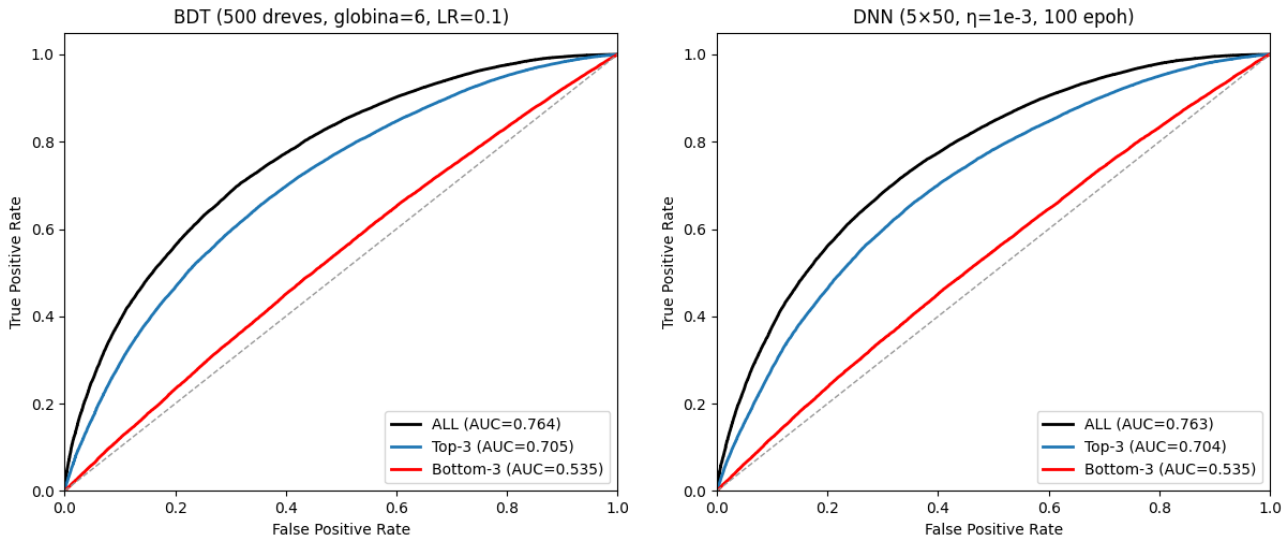
Vpliv vhodnih spremenljivk

V tem razdelku pogledamo, kako vhodne spremenljivke prispevajo k ločevanju signala od ozadja. Nekatere imajo večji vpliv na napoved, druge manjši. Pomembnost ocenimo s CatBoostovo mero `PredictionValuesChange` na validacijskem vzorcu. Ta meri povprečno spremembo napovedi, ko model uporabi določeno spremenljivko. Prispevke seštejemo po vseh drevesih in povprečimo čez vzorce; večja vrednost pomeni večjo pomembnost. Običajno so vrednosti normalizirane (npr. na vsoto 100). Spodaj so prikazane najbolj in najmanj pomembne značilke za model s 500 drevesi globine 6 in *learning rate* 0,1:



Slika 12: Najmanj in najbolj pomembne značilke glede na mero `PredictionValuesChange`.

Da bi preverili "resničnost" teh ocen, lahko tako za odločitvena drevesa (levo), kot tudi za nevronsko mrežo (desno) izračunamo AUC, v primeru, ko uporabimo vseh 18 značilk, ko uporabimo le Top 3 značilke in ko uporabimo le Bottom 3 značilke:



Slika 13: ROC krivulje za različne nabore vhodnih značilk.

Kot pričakovano, je AUC za Top 3 značilke blizu tisti, ki jo dobimo z vsemi značilkami, medtem ko je AUC za Bottom 3 značilke precej slabša.